Worked investigation: the BGL FATAL storage/TLB fault

This is summarized from a **real transcript** from a Kiro session with the `logbase` MCP server connected. Three people — an engineer, a team lead, and a manager — asked about the same BGL FATAL logs, and the agent selected and called the LogBase tools to answer each. The questions, tool calls, executed SQL (lifted verbatim from each result's `provenance`, with its `call_id`), results, and answers are reproduced below. Minor edits only: long JSON payloads are summarized to the fields that matter, and the full weekly bucket list is collapsed to its non-zero rows. Questions are kept in the order they were asked; follow-up questions appear as their own sub-sections.

Data scope. BGL dataset, `bgl_logs`. All FATAL-level work below sits inside a single active window, **2005-05-30 → 2006-01-02** (source-local); every week outside that is zero. Totals cited: 855,501 FATAL rows; 988,783 severe-level rows (SEVERE/ERROR/FATAL/FAILURE). BGL (2005–2006) and OpenStack (2017) are different systems and periods and are never merged.

Reading the counts. Every count is **physical log lines**, not incidents. This matters here: on the peak day one fault signature (`0x00004ed8`) repeated exactly **16,384 = 2^14** times, the fingerprint of one condition re-logged in a loop, not 16k separate failures. Observed facts and hypotheses are labelled separately in each role answer. Team ownership is fictional demonstration data.

Role 1 — Engineer

> ***User:*** *I'm an engineer. The BGL logs have FATAL-level entries. What's failing and in which components?*

`count_by` — FATAL by component · `call_58ddb73a54124981b52435c496f297fa`

```
WITH ownership AS (
    SELECT dataset, component, MIN(team) AS team
    FROM component_ownership
    GROUP BY dataset, component
), filtered AS (
    SELECT COALESCE(NULLIF(l.component, ''), 'Unknown') AS group_value
    FROM bgl_logs AS l
    LEFT JOIN ownership AS o
      ON o.dataset = 'bgl' AND o.component = l.component
    WHERE l.level IN ('FATAL')
), grouped AS (
    SELECT group_value, COUNT(*) AS log_count
    FROM filtered
    GROUP BY group_value
)
SELECT group_value, log_count,
       SUM(log_count) OVER () AS total_matching_rows,
       COUNT(*) OVER () AS total_groups
FROM grouped
ORDER BY log_count DESC, group_value ASC
LIMIT 100;
```

Result (`total_matching_rows` = 855,501):

Component	FATAL lines	Share
KERNEL	626,756	73.26%
APP	228,536	26.71%
LINKCARD	206	0.02%
MMCS	3	0.00%

`search` — pull FATAL evidence · `call_16eb3d846257444783b2bbebf49aecdb`

Query: `fatal OR failure OR error`, levels `["FATAL"]`, limit 10.

```
SELECT COUNT(*) AS matches FROM bgl_logs_fts
WHERE content @@ websearch_to_tsquery('fatal OR failure OR error')
  AND level IN ('FATAL');

SELECT source_file, line_id, event_time, level, component,
       LEFT(content, 500) AS content_preview
FROM bgl_logs_fts
WHERE content @@ websearch_to_tsquery('fatal OR failure OR error')
  AND level IN ('FATAL')
ORDER BY event_time, source_file, line_id LIMIT 10;
```

Result: **344,990 matches**, truncated to 10. Ranks 1–10 are all `BGL.log`, line_ids 4988–4997, 2005-06-03 16:17:40 (source-local), component APP, identical message: `ciod: Error loading /p/gb2/stella/RAPTOR/65641/raptor: invalid or missing program image, No such file or directory`.

Engineer answer (as given)

- **Observed (fact).** 855,501 FATAL rows concentrate in two components: KERNEL (626,756; 73.26%) and APP (228,536; 26.71%); LINKCARD and MMCS are negligible. A FATAL text search returned 344,990 total matches; the earliest-ordered 10 are all the APP `ciod` program-load error, near-identical and ~0.3s apart.
- **Suggestive, not proven.** The 10 APP rows repeat within ~0.3s, so they likely reflect one launch condition repeating, not 10 separate faults. This sample is APP-only and time-ordered — it is not the distribution of all FATALs, and KERNEL (the larger contributor) is not represented in it.
- **Next check (suggested, not executed).** Pull KERNEL FATAL evidence specifically, get full raw context on a ciod APP row and its neighbours, and run a FATAL trend to see whether the volume is steady or bursty.

Follow-up — most common FATAL component, with node and timestamp

> ***User:*** *Pick the most common FATAL component and show me a few real example lines with their node and timestamp.*

Most common FATAL component: **KERNEL** (626,756; 73.26%).

`search` — KERNEL FATAL sample · `call_f397a38625cf4d739284d019981e7eff`

Query: `kernel OR rts OR instruction OR interrupt`, levels `["FATAL"]`, limit 10.

```
SELECT COUNT(*) AS matches FROM bgl_logs_fts
WHERE content @@ websearch_to_tsquery('kernel OR rts OR instruction OR interrupt')
  AND level IN ('FATAL');

SELECT source_file, line_id, event_time, level, component,
       LEFT(content, 500) AS content_preview
FROM bgl_logs_fts
WHERE content @@ websearch_to_tsquery('kernel OR rts OR instruction OR interrupt')
  AND level IN ('FATAL')
ORDER BY event_time, source_file, line_id LIMIT 10;
```

Result: **403,783 matches**, truncated to 10 — all `KERNEL`, message `data TLB error interrupt`, around 2005-06-05 00:08 (source-local). The preview does not carry the node, so full details were pulled for three rows.

`get_event_details` — line 17742 · `call_11f22060b80641f7a4d001aa27af54e7`

```
WITH ownership AS (
    SELECT dataset, component, MIN(team) AS team
    FROM component_ownership
    GROUP BY dataset, component
)
SELECT l.source_file, l.line_id, l.event_time, l.level, l.component, l.content,
       l.raw_text, l.parse_ok, l.parse_error, l.event_id, l.event_template,
       l.raw_text_base64, l.label, l.epoch_seconds, l.node, l.log_type,
       COALESCE(o.team, 'Unassigned') AS owning_team
FROM bgl_logs AS l
LEFT JOIN ownership AS o
  ON o.dataset = 'bgl' AND o.component = l.component
WHERE l.source_file = 'BGL.log' AND l.line_id = 17742
ORDER BY l.source_file, l.line_id
LIMIT 2;
```

```
KERNDTLB 1117955293 2005.06.05 R20-M0-N2-C:J10-U11 2005-06-05-00.08.13.410695 R20-M0-N2-C:J10-U11 RAS KERNEL FATAL data TLB error int
```

node `R20-M0-N2-C:J10-U11` · event_time 2005-06-05 00:08:13.410695 · label `KERNDTLB` · owning_team Compute Reliability *(fictional)*.

`get_event_details` — line 17745 · `call_0bca652b349d4a9d93d37c295c2f25b3`

Same SQL shape, `line_id = 17745`.

```
KERNDTLB 1117955293 2005.06.05 R20-M0-N2-C:J14-U01 2005-06-05-00.08.13.577322 R20-M0-N2-C:J14-U01 RAS KERNEL FATAL data TLB error int
```

node `R20-M0-N2-C:J14-U01` · event_time 2005-06-05 00:08:13.577322.

`get_event_details` — line 18075 · `call_8fcc205669ed4f60961056073bd1ac1d`

Same SQL shape, `line_id = 18075`.

```
KERNDTLB 1117955303 2005.06.05 R20-M0-N7-C:J09-U11 2005-06-05-00.08.23.094716 R20-M0-N7-C:J09-U11 RAS KERNEL FATAL data TLB error int
```

node `R20-M0-N7-C:J09-U11` · event_time 2005-06-05 00:08:23.094716.

- **Observed (fact).** Three real KERNEL FATAL lines, each the `KERNDTLB` "data TLB error interrupt" message on a different R20 node (N2/J10, N2/J14, N7/J09), all within ~10s on 2005-06-05.
- **Not established.** Whether `KERNDTLB` is the single most frequent KERNEL FATAL message across all 626,756 rows — only the first time-ordered rows were inspected.

Follow-up — context before and after the first TLB interrupt

> ***User:*** *yes, I want to see what happened right before and after the first TLB interrupt on that node.*

`nearby_events` — ±10 min around line 17742 · `call_179d432e4c8b4b89bd80481d2948f31b`

Anchored at `BGL.log` line 17742, node `R20-M0-N2-C:J10-U11`, `window_minutes` 10, `limit` 20. For BGL, nearby_events defaults to the anchor's node only. The four executed statements (anchor resolve, window count, before-side, after-side) are in the provenance; the after side:

```
SELECT source_file, line_id, event_time, level, component,
       LEFT(content, 500) AS content_preview
FROM bgl_logs
WHERE event_time >= '2005-06-04 23:58:13.410695'::timestamp
  AND event_time <= '2005-06-05 00:18:13.410695'::timestamp
  AND node = 'R20-M0-N2-C:J10-U11'
  AND (event_time > '2005-06-05 00:08:13.410695'::timestamp
       OR (event_time = '2005-06-05 00:08:13.410695'::timestamp
           AND (source_file > 'BGL.log'
                OR (source_file = 'BGL.log' AND line_id > 17742))))
ORDER BY event_time ASC, source_file ASC, line_id ASC LIMIT 10;
```

Result: `total_matches` = **1**, not truncated — only the anchor itself.

Position	source_file:line	event_time	level	component	message
before	—	—	—	—	*(no events for this node in window)*
ANCHOR	BGL.log:17742	2005-06-05 00:08:13.410695	FATAL	KERNEL	data TLB error interrupt
after	—	—	—	—	*(no events for this node in window)*

- **Observed (fact).** Within ±10 min, node `R20-M0-N2-C:J10-U11` logged nothing else — the TLB interrupt is the only event for that node in the 20-minute span.
- **Boundary note.** The window is scoped to the anchor's node only; "nothing before or after" means nothing *from that node*, not that the cluster was quiet. Absence of nearby lines is not proof of anything.

Role 2 — Team lead

> ***User:*** *I'm a team lead. For BGL FATAL errors, which components and teams are involved, and how often?*

`count_by` — FATAL by team · `call_733f0c6784234dc7bc795d5696364b31`

```
WITH ownership AS (
    SELECT dataset, component, MIN(team) AS team
    FROM component_ownership
    GROUP BY dataset, component
), filtered AS (
    SELECT COALESCE(o.team, 'Unassigned') AS group_value
    FROM bgl_logs AS l
    LEFT JOIN ownership AS o
      ON o.dataset = 'bgl' AND o.component = l.component
    WHERE l.level IN ('FATAL')
), grouped AS (
    SELECT group_value, COUNT(*) AS log_count
    FROM filtered
    GROUP BY group_value
)
SELECT group_value, log_count,
       SUM(log_count) OVER () AS total_matching_rows,
       COUNT(*) OVER () AS total_groups
FROM grouped
ORDER BY log_count DESC, group_value ASC
LIMIT 100;
```

Result (`total_matching_rows` = 855,501). Each component maps to one team, so the shares match the component breakdown exactly:

Component	Owning team *(fictional)*	FATAL lines	Share
KERNEL	Compute Reliability	626,756	73.26%
APP	Application Operations	228,536	26.71%
LINKCARD	Infrastructure	206	0.02%
MMCS	Platform Operations	3	0.00%

Team-lead answer (as given)

- **Observed (fact).** Two components carry ~99.97% of FATAL volume: KERNEL (Compute Reliability, ~73%) and APP (Application Operations, ~27%). Each component maps to exactly one team, so component and team shares are identical.
- **Suggestive, not proven.** From earlier evidence the KERNEL FATALs sampled were `KERNDTLB` "data TLB error interrupt" across R20 nodes, and the APP FATALs were repeated `ciod` program-load errors — observed patterns in samples, not confirmed causes.
- **What to look at first.** Trend the FATAL volume over time; compare KERNEL vs APP across the timeline; coordinate the two owning teams; deprioritise LINKCARD/MMCS by volume.

Follow-up — weekly trend across the whole BGL period

> ***User:*** *Show me a weekly trend of those FATAL errors across the whole BGL period. Is it steady, spiking, or trailing off?*

`trend` weekly · `call_3b73552c7e494df79fca21b8528d97d8`

`interval` week, `start` 2005-01-01, `end` 2006-12-31, `level` FATAL.

```
SELECT COUNT(*) AS matches FROM bgl_logs
WHERE event_time >= '2005-01-01 00:00:00'::timestamp
  AND event_time < '2006-12-31 00:00:00'::timestamp
  AND event_time IS NOT NULL AND level = 'FATAL';

SELECT date_trunc('week', event_time) AS bucket, COUNT(*) AS count
FROM bgl_logs
WHERE event_time >= '2005-01-01 00:00:00'::timestamp
  AND event_time < '2006-12-31 00:00:00'::timestamp
  AND event_time IS NOT NULL AND level = 'FATAL'
GROUP BY bucket ORDER BY bucket;
```

Result: `total_timed_matches` = 855,501 across 105 buckets. All non-zero weeks fall in 2005-05-30 → 2006-01-02; every other week is zero. Non-zero rows:

Week of (source-local)	FATAL lines
2005-05-30	6,262
2005-06-06	154,761
2005-06-13	380,987
2005-06-20	4,508
2005-06-27	3,246
2005-07-04	4,274
2005-07-11	7,565
2005-07-18	5,592
2005-07-25	1,137
2005-08-01	2,313
2005-08-08	31,411
2005-08-15	14,934
2005-08-22	14,699
2005-08-29	24,784
2005-09-05	19,337
2005-09-12	7,692
2005-09-19	27,432
2005-09-26	11,350
2005-10-03	8,746
2005-10-10	11,544
2005-10-17	8,723
2005-10-24	10,844
2005-10-31	6,089
2005-11-07	6,772
2005-11-14	13,042
2005-11-21	15,381
2005-11-28	13,656
2005-12-05	22,075
2005-12-12	10,113
2005-12-19	2,690
2005-12-26	2,934
2006-01-02	608

- **Observed (fact).** The shape is a sharp early spike then a long decline. The weeks of 2005-06-06 (154,761) and 2005-06-13 (380,987) together hold ~536k rows — about 63% of all FATAL volume in two weeks; 2005-06-13 is the peak. A long tail of lower volume (mostly ~5k–30k/week) runs through summer/fall with smaller bumps (08-08, 09-19, 12-05), trailing to 608 by 2006-01-02.
- **Boundary notes.** Counts are log lines, not incidents; a zero week means no FATAL lines recorded, not a healthy or running system. The 2006 flat stretch likely reflects end of data coverage. First/last active buckets may be partial; times are source-local, no timezone conversion.

Follow-up — distinctive words, worst week vs calmer week

> ***User:*** *Can you also compare the words that were unusually common during the worst week against a calmer week? I want a lead on what changed.*

The tool caps each window at 24 hours, so this compares a **peak day (2005-06-13)** against a **calm day (2005-10-17)**, both FATAL-only — a representative slice, not the full weeks.

`distinctive_terms` — peak day vs calm day · `call_06e4d393319d490b816cadf1a35a44ce`

foreground 2005-06-13 → 2005-06-14, baseline 2005-10-17 → 2005-10-18, levels `["FATAL"]`, top_n 15.

```
WITH fg_terms AS (
    SELECT unnest(ts_dict_agg(content)) AS term,
           unnest(ts_dict_count(content)) AS docs
    FROM bgl_logs_fts WHERE event_time >= '2005-06-13 00:00:00'::timestamp AND event_time < '2005-06-14 00:00:00'::timestamp AND ever
), bg_terms AS (
    SELECT unnest(ts_dict_agg(content)) AS term,
           unnest(ts_dict_count(content)) AS docs
    FROM bgl_logs_fts WHERE event_time >= '2005-10-17 00:00:00'::timestamp AND event_time < '2005-10-18 00:00:00'::timestamp AND ever
), fg_total AS (
    SELECT COUNT(*) AS n FROM bgl_logs_fts WHERE event_time >= '2005-06-13 00:00:00'::timestamp AND event_time < '2005-06-14 00:00:0
), bg_total AS (
    SELECT COUNT(*) AS n FROM bgl_logs_fts WHERE event_time >= '2005-10-17 00:00:00'::timestamp AND event_time < '2005-10-18 00:00:0
), joined AS (
    SELECT f.term AS term,
           f.docs AS foreground_docs,
           COALESCE(b.docs, 0) AS baseline_docs,
           (SELECT n FROM fg_total) AS foreground_total,
           (SELECT n FROM bg_total) AS baseline_total
    FROM fg_terms f
    LEFT JOIN bg_terms b ON b.term = f.term
    WHERE f.docs >= 5
      AND f.term NOT IN ('the', 'a', 'an', 'and', 'or', 'of', 'to', 'in', 'on', 'at', 'for', 'is', 'as', 'by', 'with', 'from')
), scored AS (
    SELECT term, foreground_docs, baseline_docs,
           foreground_total, baseline_total,
           foreground_docs::double precision / foreground_total AS foreground_share,
           CASE WHEN baseline_total > 0
                THEN baseline_docs::double precision / baseline_total
                ELSE 0 END AS baseline_share
    FROM joined
    WHERE foreground_total > 0
), qualifying AS (
    SELECT term, foreground_docs, baseline_docs,
           foreground_total, baseline_total,
           foreground_share, baseline_share,
           foreground_share - baseline_share AS share_change,
           foreground_docs - foreground_total * baseline_share AS excess_documents,
           CASE WHEN baseline_share > 0
                THEN foreground_share / baseline_share ELSE NULL END AS ratio
    FROM scored
    WHERE foreground_docs - foreground_total * baseline_share > 0
)
SELECT term, foreground_docs, baseline_docs,
       foreground_total, baseline_total,
       foreground_share, baseline_share,
       share_change, excess_documents, ratio,
       COUNT(*) OVER () AS total_qualifying_terms
FROM qualifying
ORDER BY excess_documents DESC, foreground_docs DESC, term ASC
LIMIT 15;
```

Result: foreground_total 33,921; baseline_total 1,077; 43 qualifying terms, 15 returned. Every term below is new in the comparison (baseline share 0, ratio undefined):

Term	Peak-day share	Calm-day share	Peak docs	Excess docs
instruction	48.3299%	0%	16,394	16,394
data	48.324%	0%	16,392	16,392
interrupt	48.324%	0%	16,392	16,392
address	48.3182%	0%	16,390	16,390
0x00004ed8	48.3005%	0%	16,384	16,384
storage	48.3005%	0%	16,384	16,384
failed	3.0453%	0%	1,033	1,033
read	3.007%	0%	1,020	1,020
control	2.9981%	0%	1,017	1,017
stream	2.9981%	0%	1,017	1,017

(Remaining returned terms: `0` 42 docs, `gb2` 32, `p` 32, `1` 24, `miranda.dbg` 24.)

- **Observed (fact).** On the peak day a specific storage-address signature appears that is absent from the calm day: the top six terms (`instruction`, `data`, `interrupt`, `address`, `0x00004ed8`, `storage`) sit at ~48% share with near-identical doc counts (~16,384–16,394), i.e. one repeated message template. The literal hex address `0x00004ed8` appears exactly 16,384 = 2^14 times. A second smaller cluster (`failed`/`read`/`control`/`stream`, ~3% share, ~1,017–1,033 docs) is also new.
- **Suggestive, not proven.** These terms are investigative leads, not a cause; co-occurrence does not prove one message caused another. The 2^14 repeat strongly suggests one looping fault, so volume overstates distinct incidents.
- **Boundary note.** One peak day vs one calm day (24h cap), not the full weeks. Baseline share 0 means the term was absent from that calm day's FATALs, not the whole fall period.

Role 3 — Manager

> ***User:*** *You are now talking to a manager. Across the whole BGL corpus, which teams and components carry the most severe problems, and what should we prioritise? Keep it concise.*

`top_problems` — severe-level by component · `call_e8b5533977b14ed48d948e2b0a532dc9`

`group_by` component, levels `["SEVERE","ERROR","FATAL","FAILURE"]`, top_n 10.

```
WITH ownership AS (
    SELECT dataset, component, MIN(team) AS team
    FROM component_ownership
    GROUP BY dataset, component
), filtered AS (
    SELECT COALESCE(l.component, 'Unknown') AS component,
           COALESCE(o.team, 'Unassigned') AS team
    FROM bgl_logs AS l
    LEFT JOIN ownership AS o
      ON o.dataset = 'bgl' AND o.component = l.component
    WHERE l.level IN ('SEVERE', 'ERROR', 'FATAL', 'FAILURE')
), grouped AS (
    SELECT component AS group_value, team AS owning_team, COUNT(*) AS log_count
    FROM filtered
    GROUP BY component, team
), ranked AS (
    SELECT group_value, owning_team, log_count,
           SUM(log_count) OVER () AS total_matching_rows,
           COUNT(*) OVER () AS total_groups
    FROM grouped
)
SELECT group_value, owning_team, log_count, total_matching_rows, total_groups
FROM ranked
ORDER BY log_count DESC, group_value ASC, owning_team ASC
LIMIT 10;
```

Result (`total_matching_rows` = 988,783, `truncated: false`, 9 groups):

Rank	Component	Owning team *(fictional)*	Records	Share
1	KERNEL	Compute Reliability	626,756	63.39%
2	APP	Application Operations	228,536	23.11%
3	MMCS	Platform Operations	88,532	8.95%
4	DISCOVERY	Platform Operations	42,149	4.26%
5	MONITOR	Platform Operations	1,681	0.17%
6	HARDWARE	Infrastructure	873	0.09%
7	LINKCARD	Infrastructure	206	0.02%
8	BGLMASTER	Platform Operations	49	0.00%
9	SERV_NET	Infrastructure	1	0.00%

`top_problems` — FATAL by team · `call_67fce143b38a42deb146dc4e885b0df8`

`group_by` team, levels `["FATAL"]`, top_n 10.

```
WITH ownership AS (
    SELECT dataset, component, MIN(team) AS team
    FROM component_ownership
    GROUP BY dataset, component
), filtered AS (
    SELECT COALESCE(l.component, 'Unknown') AS component,
           COALESCE(o.team, 'Unassigned') AS team
    FROM bgl_logs AS l
    LEFT JOIN ownership AS o
      ON o.dataset = 'bgl' AND o.component = l.component
    WHERE l.level IN ('FATAL')
), grouped AS (
    SELECT team AS group_value, COUNT(*) AS log_count
    FROM filtered
    GROUP BY team
), ranked AS (
    SELECT group_value, log_count,
           SUM(log_count) OVER () AS total_matching_rows,
           COUNT(*) OVER () AS total_groups
    FROM grouped
)
SELECT group_value, log_count, total_matching_rows, total_groups
FROM ranked
ORDER BY log_count DESC, group_value ASC
LIMIT 10;
```

Result (`total_matching_rows` = 855,501, `truncated: false`):

Rank	Team *(fictional)*	FATAL lines	Share
1	Compute Reliability	626,756	73.26%
2	Application Operations	228,536	26.71%
3	Infrastructure	206	0.02%
4	Platform Operations	3	0.00%

Manager answer (as given)

- **Observed (fact).** Across severe-level records (988,783), volume concentrates in KERNEL/Compute Reliability (63.39%) and APP/Application Operations (23.11%), then MMCS (8.95%) and DISCOVERY (4.26%), both Platform Operations; the rest are <1% combined. For FATAL specifically, Compute Reliability holds 73.26% and Application Operations 26.71%.
- **Recommended priorities.** (1) Compute Reliability (KERNEL) — ~63% severe / ~73% FATAL, tied to the June 2005 storage-address/TLB signature. (2) Application Operations (APP) — ~23% severe / ~27% FATAL, repeated `ciod` program-load failures. (3) Platform Operations (MMCS+DISCOVERY) — ~13% severe but near-zero FATAL; monitor. (4) Infrastructure — negligible; watch only.
- **Decision caveats.** These rank log-record volume, not validated incidents, business impact, or confirmed cause; a single fault emits many near-identical lines (one 2^14 repeat observed), so volume overstates distinct problems — especially for KERNEL. The severe total is time-concentrated (~63% of FATAL in two weeks of June 2005). Team labels are fictional demonstration data.

Follow-up — confidence, limits, and the single top priority

> ***User:*** *How confident are we that these counts mean real incidents? What are we not able to tell from the logs alone? Also, if you had to name the single top priority and one sentence on why, what would it be?*

No new tool calls; this answer draws on the results already gathered above.

- **Confidence (fact + reasoning).** Low confidence that counts equal incidents. Every tool counts physical log rows, and here volume demonstrably overstates incidents: `0x00004ed8` repeated exactly 16,384 = 2^14 times on the peak day; APP ciod errors were near-identical lines ~0.3s apart; ~63% of FATAL volume hit two weeks of June 2005. The counts are reliable as log-record volume and relative hotspots, not as incident counts.
- **Not establishable from logs alone.** Incident count; impact/blast radius (jobs lost, downtime, SLA); root cause and causality; whether anything was resolved (a zero week can be fix, offline node, or end of coverage — BGL data stops early Jan 2006); real ownership (labels are fictional); coverage gaps (a zero bucket is not proof of health).
- **Single top priority.** Compute Reliability's KERNEL storage-address/TLB fault from the June 2005 spike — it is ~63% of severe records and ~73% of FATALs, concentrated in a single two-week episode, making it both the largest and most contained target to investigate first.

What this investigation establishes — and what it does not

- **Established:** BGL FATAL volume (855,501 rows) concentrates in KERNEL (Compute Reliability, 73.26%) and APP (Application Operations, 26.71%); across severe-level records (988,783) the top four are KERNEL, APP, MMCS, DISCOVERY; the FATAL volume is time-bound to 2005-05-30 → 2006-01-02 and spikes hardest the weeks of 2005-06-06 and 2005-06-13 (peak 380,987); on the peak day a storage-address signature (`0x00004ed8`, 16,384 docs) dominates and is absent from the calm day; real example KERNEL FATAL lines are `KERNDTLB` "data TLB error interrupt" on R20 nodes around 2005-06-05.
- **Not established:** how many distinct incidents produced these lines; any job, availability, or cost impact; why the TLB/storage-address faults occurred; whether they were resolved. Temporal proximity and shared signatures are correlation, not a proven cause. Log-line counts are not incident counts — the 2^14 repeat shows the gap can be large. Team ownership is fictional demonstration data.

Tools used

`count_by` (FATAL by component, FATAL by team), `search` (full-text + level filter), `get_event_details` (identity lookups for lines 17742, 17745, 18075), `nearby_events` (identity-anchored, node-scoped context), `trend` (weekly, whole period), `distinctive_terms` (peak day vs calm day), `top_problems` (severe by component, FATAL by team).

Every result carried a `provenance` block (`call_id`, arguments, UTC execution timestamps, exact SQL); the `call_id`s above are the real ones from the session.